Machine Learning

Core Theory Notes

① Regression vs Classification

② Linear vs Regularised

③ Overfitting & Underfitting

④ RMSE, MAE & R^2

⑤ Cross-Validation

Supervised Learning — Regression Deep Dive • Final Examination Preparation
Ames Housing Project — PropTech Valuation Context (NoBroker / MagicBricks)

Q1 — What is Regression? How is it different from Classification?

Definition of Regression

Regression is a **supervised machine learning technique** used to predict a **continuous numerical output** (a real-valued quantity). The model learns a mapping f(X) → ŷ by minimising a loss function (typically Mean Squared Error) on labelled training examples.

Objective: minimise MSE = $\frac{1}{n} \times \Sigma(y_i - \hat{y}_i)^2$

where y_i = actual value, $\hat{y}_i$ = predicted value, n = number of samples

Definition of Classification

Classification predicts a **discrete class label** from a finite set of categories. The model learns decision boundaries that separate data points into distinct groups, minimising a classification loss (typically Cross-Entropy).

The Core Distinction

- **Regression** → Output is a **NUMBER** (salary, temperature, property price, crop yield)
- **Classification** → Output is a **CATEGORY** (spam / not-spam, approved / rejected)

Comparison Table

Aspect	Regression	Classification
Output Type	Continuous real number	Discrete class label
Example Output	₹68,40,000 property price	APPROVED / REJECTED
Loss Function	MSE / RMSE / MAE	Cross-Entropy / Log Loss
Evaluation	R^2, RMSE, MAE	Accuracy, Precision, Recall, F1
Decision Boundary	No fixed boundary — fits a regression curve or surface	Learns a separating hyperplane or curved boundary
Common Algorithms	Linear Reg., Ridge, Lasso, XGBoost Regressor	Logistic Reg., SVM, Random Forest Classifier, Naïve Bayes

Real-World Indian Examples

Example 1 — HDFC Bank: EMI Amount Prediction (Regression)

HDFC's home loan engine collects applicant features — monthly income, CIBIL credit score, loan tenure, LTV ratio, property valuation — and predicts the exact monthly EMI (a continuous number in ₹). Since EMI can be ₹12,500 or ₹87,430.50, this is unmistakably a **Regression** problem.

- Features (X): Monthly Income, CIBIL Score, Loan Amount, Tenure, Age

- Target (y): EMI in ₹ per month (e.g., ₹24,350.75 / month)

Example 2 — Maharashtra Agriculture: Crop Yield Prediction (Regression)

The Maharashtra Government and ICAR use satellite remote-sensing data, soil health cards, rainfall records, and fertiliser usage to predict wheat / soybean yield in **quintals/hectare** for Vidarbha and Marathwada farmers. This continuous output guides MSP procurement and mandi supply planning.

- Features (X): Rainfall (mm), Soil NPK index, Temperature, Sowing Date, Crop Variety
- Target (y): Yield in quintals/hectare (e.g., 18.6 q/ha)

Counter-example (Classification): SBI loan default detection — the output is binary: DEFAULT (1) or NO DEFAULT (0). Classification models (Logistic Regression, XGBoost Classifier) are used here, not regression.

Q2 — Simple Linear Regression vs Multiple Linear Regression vs Regularised Regression

Simple Linear Regression (SLR)

Models the linear relationship between **one independent variable (feature)** and one dependent variable (target). It finds the best-fit straight line through the scatter of data points by minimising the Residual Sum of Squares.

$$y = \beta_0 + \beta_1 \times X + \varepsilon$$

β_0 = intercept (y-axis crossing), β_1 = slope (rate of change), ε = irreducible error term

- **Example:** Predicting exam marks (y) purely from hours studied (X).
- **Limitation:** Real-world outcomes almost never depend on just a single variable.

Multiple Linear Regression (MLR)

Extends SLR to **two or more independent variables**. The model fits a hyperplane (multi-dimensional flat surface) through the data. Each coefficient β_i represents the marginal effect of feature X_i on y, holding all other features constant.

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ + \beta_p X_p + \varepsilon$$

House Price = β_0 + β_1×GrLivArea + β_2×OverallQual + β_3×YearBuilt + …

- **Example:** HDFC property valuation with 79 features — the Ames Housing problem.
- **Problem:** When p n or features are strongly correlated (multicollinearity), OLS coefficients become **unstable and blow up** — the model overfits noise.

Regularised Regression — Ridge (L2) & Lasso (L1)

Regularisation adds a **penalty term** to the OLS loss that shrinks large coefficients toward zero, reducing model complexity and variance (overfitting) at the cost of a small increase in bias. This is the classical **Bias–Variance Trade-off**.

$$\textbf{Ridge Loss} = \Sigma(y_i - \hat{y}_i)^2 + \lambda \times \Sigma\beta_i^2 \textbf{ (L2 norm penalty)}$$

Lasso Loss = $\Sigma(y_i - \hat{y}_i)^2 + \lambda \times \Sigma|\beta_i|$ (L1 norm penalty)

Property	Ridge Regression (L2)	Lasso Regression (L1)
Penalty	Squared coefficients ($\|\|\beta\|\|^2$)	Absolute values of coefficients ($\|\|\beta\|\|_1$)
Coefficient Behaviour	All shrink toward zero; none become exactly 0	Forces some to exactly 0 — automatic feature selection
Sparsity	Dense — all features retained	Sparse — irrelevant features eliminated
Best For	Multicollinearity; many small correlated effects	High-dimensional data; automated feature pruning

Property	Ridge Regression (L2)	Lasso Regression (L1)
Hyperparameter Tuning	RidgeCV auto-selects optimal λ	LassoCV auto-selects optimal λ
Project Result (Ames)	RMSE ≈ $20,497 (λ = 10)	RMSE ≈ $20,058 (λ = 0.001, 123 features → 0)

Key Insight: When λ = 0, regularised regression reduces to plain OLS. As λ → ∞, all coefficients shrink to 0 (extreme underfitting). The optimal λ is found by Cross-Validation.

Q3 — Explain Overfitting and Underfitting. How does Regularisation (L1/L2) help?

Underfitting — High Bias

The model is **too simple** to capture the true relationship in the data. It performs poorly on **both training and test sets**. Like a student who barely studied — fails both mock exams and the final.

Overfitting — High Variance

The model **memorises training data noise** instead of learning the underlying pattern. It performs excellently on training data but **fails on unseen test data**. Like a student who only memorised previous-year papers — helpless when a new question pattern appears.

Characteristic	Underfitting (High Bias)	Overfitting (High Variance)
Training Error	HIGH (cannot fit even training data)	Very LOW (memorises training data)
Test Error	HIGH	HIGH (fails to generalise)
Root Cause	Model too simple / too few features	Model too complex / too many features / noisy data
Visual Sign	Flat line through curved data	Wiggly curve passing through every point
Analogy	Under-prepared student	Rote-memoriser student
Fix	More features, polynomial terms, complex model	Regularisation, more training data, cross-validation

The Bias–Variance Trade-Off

Total Prediction Error = Bias² + Variance + Irreducible Noise

Goal: simultaneously minimise Bias² (underfitting) AND Variance (overfitting)

How Regularisation (L1/L2) Prevents Overfitting

Mechanism: Regularisation modifies the loss function by adding a penalty on coefficient magnitudes, forcing the optimiser to keep weights small and the model simple.

Ridge (L2) — Coefficient Shrinkage:

- Penalises the **squared** values of coefficients.
- Spreads weight evenly across correlated features — no coefficient is zeroed.
- Geometrically: the L2 constraint region is a smooth sphere; optimal solution is a smooth interior point.

Lasso (L1) — Sparse Feature Selection:

- Penalises the **absolute** values of coefficients.
- The L1 constraint region is a diamond with sharp corners on the axes.

- The optimal solution often lands **exactly on a corner**, forcing some β = 0.
- Result: automatic feature selection — only the most predictive features survive.

Effect of λ (Regularisation Strength):

- λ = 0 → Plain OLS regression (may overfit if data is noisy)
- λ = small → Mild regularisation (sweet spot)
- λ = large → All coefficients → 0 (underfitting)
- Optimal λ found via k-Fold Cross-Validation (RidgeCV / LassoCV)

Project Evidence (Ames Housing): Plain OLS achieved Test RMSE = $22,630. Ridge regularisation improved this to $20,497. Lasso further improved to $20,058 by pruning 123 redundant/collinear features, confirming that regularisation meaningfully reduces overfitting and coefficient instability.

Q4 — Define RMSE, MAE, and R^2. When would you prefer RMSE over MAE?

Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \times \Sigma \, |y_i - \hat{y}_i|$$

Average of absolute differences between actual and predicted values

- Measures the **average absolute deviation** in the same units as the target (e.g., ₹).
- **Robust to outliers** — each error contributes equally regardless of magnitude.
- Interpretable: "On average, predictions are off by ₹14,162."
- Corresponds to **median-optimal prediction** (minimising MAE gives median prediction).

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{[\, \frac{1}{n} \times \Sigma (y_i - \hat{y}_i)^2 \,]}$$

Square root of the average squared differences — same units as target

- **Squares errors** before averaging — larger errors are penalised **much more heavily**.
- Sensitive to outliers; a few large errors can dominate the RMSE value.
- Corresponds to **mean-optimal prediction** (minimising RMSE gives mean prediction).
- **Always: RMSE ≥ MAE**. When RMSE MAE, large outlier errors are present.

R^2 — Coefficient of Determination

$$R^2 = 1 - [\, \Sigma (y_i - \hat{y}_i)^2 \div \Sigma (y_i - \bar{y})^2 \,]$$

$\bar{y}$ = mean of actual values | $R^2 = 0.93$ means model explains 93% of variance in y

- **Scale-free** measure, ranges from $-\infty$ to 1.0 (higher is better).
- $R^2 = 1.0$ → perfect predictions. $R^2 = 0$ → no better than predicting the mean. $R^2 < 0$ → worse than mean.
- Does **not** penalise model complexity — use **Adjusted R^2** when comparing models with different feature counts.

When to Prefer RMSE over MAE?

Scenario	Preferred Metric	Reasoning
Large outliers are rare but **extremely costly** (e.g., property price error of ₹50L)	RMSE	Quadratic penalty forces model to minimise catastrophic mispricings

Scenario	Preferred Metric	Reasoning
Data is **noisy** with many natural outliers (e.g., IoT sensor readings)	MAE	Equal treatment of all errors; outliers do not dominate the metric
Business cost of error grows **non-linearly** (e.g., HDFC loan default risk)	RMSE	Squared penalty aligns with asymmetric real-world cost functions
You want a **median-optimal** prediction (e.g., income distribution with skew)	MAE	Minimising MAE yields the conditional median — more robust for skewed targets
Comparing models or tuning hyperparameters	Both	RMSE for sensitivity to large errors; MAE for robust overall performance view

*PropTech Context (NoBroker / HDFC): RMSE is preferred for property price prediction because a ₹50 lakh over-valuation is **catastrophically worse** than a ₹5 lakh error for a homebuyer taking a 20-year mortgage. RMSE's squared penalty forces the model to avoid such extreme misvaluations. Our Ames Housing project: RMSE = $19,636 vs MAE = $14,162 — the gap indicates a few expensive homes are harder to predict precisely.*

Q5 — What is Cross-Validation (k-Fold)? Why preferred over a single train-test split?

What is Cross-Validation?

Cross-Validation (CV) is a **statistical resampling technique** that evaluates how well a machine learning model generalises to **independent, unseen data**. Instead of relying on a single arbitrary train-test split, CV systematically rotates the holdout set to produce a robust, low-variance estimate of true model performance.

k-Fold Cross-Validation — Step-by-Step

Step-by-Step Process:

Step 1: Shuffle the full dataset randomly.

Step 2: Divide dataset into **k equal-sized folds** (subsets). Standard choice: k = 5 or 10.

Step 3: For each fold i (from 1 to k): use fold i as **Validation set**; train on remaining k−1 folds; record validation score (e.g., RMSE).

Step 4: Final CV Score = **Mean ± Std Deviation** of all k validation scores.

Visual Fold Diagram (k = 5):

Fold 1: [TEST] [train] [train] [train] [train]

Fold 2: [train] [TEST] [train] [train] [train]

Fold 3: [train] [train] [TEST] [train] [train]

Fold 4: [train] [train] [train] [TEST] [train]

Fold 5: [train] [train] [train] [train] [TEST]

$$\text{CV RMSE} = \text{Mean}\left(\text{RMSE}_1, \text{RMSE}_2, \text{RMSE}_3, \text{RMSE}_4, \text{RMSE}_5 \right) \pm \text{Std Dev}$$

Why Preferred Over a Single Train-Test Split?

Issue	Single Train-Test Split	k-Fold Cross-Validation
Data Efficiency	~20% of data wasted for testing; never used for training.	Every data point used for BOTH training AND validation across different folds. Maximum utilisation.
Score Variance / Reliability	Score heavily depends on which rows ended up in test set (luck of the draw). High variance.	Averages over k independent evaluations. Statistically robust, low-variance performance estimate.
Overfitting Detection	One lucky split can give falsely optimistic results.	Harder to overfit across k different validation sets. Better true generalisation estimate.
Hyperparameter Tuning	Tuning to a single test set causes data leakage (you effectively train on the test set).	CV provides uncontaminated evaluation for hyperparameter selection (RandomizedSearchCV).

Issue	Single Train-Test Split	k-Fold Cross-Validation
Small Datasets	Losing 20% to testing is very costly when data is scarce.	k-Fold makes full use of limited data — critical for medical/agricultural datasets.
Reproducibility	Results change with random seed / split choice.	With shuffle=True and fixed random_state, results are stable and reproducible.

Choosing k:

- **k = 5 or 10** → Standard choice; good balance of computation vs. estimate reliability.
- **k = n (LOOCV — Leave-One-Out CV)** → Maximum data use; computationally expensive; high variance in the estimate.
- **Stratified k-Fold** → For classification; maintains class label proportion in each fold.
- **Repeated k-Fold** → Runs k-Fold multiple times with different shuffles; even more stable estimate.

Our Project (Ames Housing): 5-Fold CV on Tuned XGBoost gave Log RMSE = 0.1197 ± small σ. This closely aligned with the holdout test RMSE ($19,636), confirming no overfitting and that the model generalises well beyond the 80% training set.

Quick Reference Summary — All 5 Questions

#	Topic	Core Idea	Key Formula / Rule of Thumb
Q 1	Regression vs Classification	Regression → continuous number output. Classification → discrete category label.	Output is a number → Regression. Output is a class → Classification.
Q 2	SLR vs MLR vs Regularised	SLR: one feature. MLR: many features. Regularised: adds penalty λ to control coefficient magnitude.	Ridge: L2 penalty (all β shrink). Lasso: L1 penalty (some β → 0). Optimal λ via CV.
Q 3	Overfitting vs Underfitting	Underfitting: model too simple, high bias. Overfitting: model memorises noise, high variance.	Total Error = Bias² + Variance + Noise. Balance via regularisation + CV.
Q 4	RMSE, MAE & R^2	RMSE penalises large errors quadratically. MAE treats all errors equally. R^2 measures explained variance.	RMSE ≥ MAE always. $R^2 = 1$ → perfect. Prefer RMSE when large errors are very costly.
Q 5	Cross-Validation	Rotate holdout fold k times. Average k scores for a robust, low-variance evaluation.	k = 5 or 10 (standard). CV Score = Mean ± Std of k validation fold scores.